Roteiro — Aula 07 (23-10): JWT e Segurança de APIs

Autenticação com JWT + cookie httpOnly (NestJS)

> Guia para escrever ao vivo. Partimos do `backend-final` (NestJS + TypeORM + Postgres). Vamos criar login/registro com **JWT**, guardar o token em um **cookie httpOnly** e proteger as rotas lendo o **sub do usuário no servidor**.

Preparação: copie o projeto para a pasta de hoje e trabalhe no backend. Suba o Postgres (`docker compose up -d db`) — o mesmo da aula de ORM.

Conceitos (rápido)

- **JWT** = um token assinado no formato `header.payload.signature`. O *payload* carrega "claims", entre elas o `sub` (o id do usuário) e o `exp` (expiração). O servidor **assina** com um segredo e **valida** a assinatura em cada requisição — sem guardar sessão (stateless).
- **Por que cookie httpOnly?** Guardar o token num cookie `httpOnly` impede que o JavaScript da página leia o token (defesa contra **XSS**). `sameSite` reduz **CSRF** e `secure` exige HTTPS. O cliente nunca precisa manipular o token.
- **sub no servidor:** depois do login, quem diz "qual usuário é" é o **token** (validado no servidor), não o cliente. Lemos `sub` do token e sabemos quem é.

---

1. Instalar dependências

```
npm install @nestjs/jwt @nestjs/passport passport passport-jwt bcryptjs cookie-parser
npm install -D @types/passport-jwt @types/bcryptjs @types/cookie-parser
```

- `@nestjs/jwt` assina/valida o token · `passport-jwt` lê e valida nas rotas
- `bcryptjs` faz o hash da senha · `cookie-parser` lê os cookies

---

2. Usuário (entidade + migration)

`src/users/entities/user.entity.ts`:

```
import { Column, Entity, PrimaryGeneratedColumn } from 'typeorm';

@Entity('usuarios')
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ unique: true })
  email: string;

  @Column()
  senha: string; // guardamos só o HASH, nunca a senha pura
}
```

Inclua a entidade no `src/data-source.ts` (array `entities`):

```
import { User } from './users/entities/user.entity';
// ...
entities: [Tarefa, User],
```

Gere e rode a migration:

```
npm run migration:generate -- src/migrations/CreateUsers
npm run migration:run
```

Gera o `CREATE TABLE "usuarios"` (id, email UNIQUE, senha).

---

3. UsersModule e UsersService

`src/users/users.service.ts`:

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { User } from './entities/user.entity';

@Injectable()
export class UsersService {
  constructor(
    @InjectRepository(User) private readonly repo: Repository&lt;User&gt;,
  ) {}

  findByEmail(email: string): Promise&lt;User | null&gt; {
    return this.repo.findOneBy({ email });
  }

  create(email: string, senhaHash: string): Promise&lt;User&gt; {
    const user = this.repo.create({ email, senha: senhaHash });
    return this.repo.save(user);
  }
}
```

`src/users/users.module.ts`:

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { User } from './entities/user.entity';
import { UsersService } from './users.service';

@Module({
  imports: [TypeOrmModule.forFeature([User])],
  providers: [UsersService],
  exports: [UsersService],
})
export class UsersModule {}
```

---

4. AuthService (registro, validação, login)

`src/auth/auth.service.ts`:

```
import { ConflictException, Injectable, UnauthorizedException } from '@nestjs/common';
import { JwtService } from '@nestjs/jwt';
import * as bcrypt from 'bcryptjs';
import { UsersService } from '../users/users.service';

@Injectable()
export class AuthService {
  constructor(
    private readonly usersService: UsersService,
    private readonly jwtService: JwtService,
  ) {}

  async register(email: string, senha: string) {
    if (await this.usersService.findByEmail(email)) {
      throw new ConflictException('E-mail já cadastrado');
    }
    const hash = await bcrypt.hash(senha, 10);
    const user = await this.usersService.create(email, hash);
    return { id: user.id, email: user.email };
  }

  async validarUsuario(email: string, senha: string) {
    const user = await this.usersService.findByEmail(email);
    if (!user || !(await bcrypt.compare(senha, user.senha))) {
      throw new UnauthorizedException('Credenciais inválidas');
    }
    return user;
  }

  async login(email: string, senha: string) {
    const user = await this.validarUsuario(email, senha);
    const payload = { sub: user.id, email: user.email }; // sub = id do usuário
    const token = await this.jwtService.signAsync(payload);
    return { token, user: { id: user.id, email: user.email } };
  }
}
```

DTO de credenciais — `src/auth/dto/auth.dto.ts`:

```
import { IsEmail, IsString, MinLength } from 'class-validator';

export class AuthDto {
  @IsEmail({}, { message: 'E-mail inválido.' })
  email: string;

  @IsString()
  @MinLength(6, { message: 'A senha deve ter ao menos 6 caracteres.' })
  senha: string;
}
```

---

5. Estratégia JWT (lê o token do cookie) e o Guard

`src/auth/jwt.strategy.ts` — aqui é onde o **servidor lê o sub do token**:

```
import { Injectable } from '@nestjs/common';
import { PassportStrategy } from '@nestjs/passport';
import { ExtractJwt, Strategy } from 'passport-jwt';
import { Request } from 'express';

@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor() {
    super({
      // Lê o token do COOKIE httpOnly (não do header Authorization).
      jwtFromRequest: ExtractJwt.fromExtractors([
        (req: Request) =&gt; req?.cookies?.access_token ?? null,
      ]),
      ignoreExpiration: false,
      secretOrKey: process.env.JWT_SECRET || 'segredo-da-aula',
    });
  }

  // O retorno vira req.user. Expomos o sub (id) e o email lidos do token.
  async validate(payload: { sub: number; email: string }) {
    return { userId: payload.sub, email: payload.email };
  }
}
```

`src/auth/jwt-auth.guard.ts`:

```
import { Injectable } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';

@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {}
```

---

6. AuthController (cookie httpOnly) e AuthModule

`src/auth/auth.controller.ts` — o **cookie é definido pelo servidor**:

```
import { Body, Controller, Get, Post, Req, Res, UseGuards } from '@nestjs/common';
import { Request, Response } from 'express';
import { AuthService } from './auth.service';
import { AuthDto } from './dto/auth.dto';
import { JwtAuthGuard } from './jwt-auth.guard';

@Controller('auth')
export class AuthController {
  constructor(private readonly authService: AuthService) {}

  @Post('register')
  register(@Body() dto: AuthDto) {
    return this.authService.register(dto.email, dto.senha);
  }

  @Post('login')
  async login(@Body() dto: AuthDto, @Res({ passthrough: true }) res: Response) {
    const { token, user } = await this.authService.login(dto.email, dto.senha);
    res.cookie('access_token', token, {
      httpOnly: true,   // o JS do navegador NÃO lê o cookie (anti-XSS)
      sameSite: 'lax',  // não envia cross-site (ajuda contra CSRF)
      secure: false,    // em produção (HTTPS): true
      maxAge: 24 * 60 * 60 * 1000, // 1 dia
    });
    return { user };
  }

  @Post('logout')
  logout(@Res({ passthrough: true }) res: Response) {
    res.clearCookie('access_token');
    return { ok: true };
  }

  @UseGuards(JwtAuthGuard)
  @Get('me')
  me(@Req() req: Request) {
    return req.user; // { userId, email } — lido do token no cookie
  }
}
```

src/auth/auth.module.ts:

```
import { Module } from '@nestjs/common';
import { JwtModule } from '@nestjs/jwt';
import { PassportModule } from '@nestjs/passport';
import { UsersModule } from '../users/users.module';
import { AuthController } from './auth.controller';
import { AuthService } from './auth.service';
import { JwtStrategy } from './jwt.strategy';

@Module({
  imports: [
    UsersModule,
    PassportModule,
    JwtModule.register({
      secret: process.env.JWT_SECRET || 'segredo-da-aula',
      signOptions: { expiresIn: '1d' },
    }),
  ],
  controllers: [AuthController],
  providers: [AuthService, JwtStrategy],
})
export class AuthModule {}
```

7. Ligar tudo e proteger as tarefas

`src/app.module.ts` — importe os novos módulos:

```
import { UsersModule } from './users/users.module';
import { AuthModule } from './auth/auth.module';
// ...
imports: [
  TypeOrmModule.forRoot(dataSourceOptions),
  TarefasModule,
  UsersModule,
  AuthModule,
],
```

`src/main.ts` — habilite a leitura de cookies:

```
import cookieParser from 'cookie-parser';
// ...
app.use(cookieParser());
```

> Atenção ao import: com `esModuleInterop: true`, use `import cookieParser from 'cookie-parser'` (import default), e **não** `import * as cookieParser`.

Proteja o `TarefasController` — exige token válido em todas as rotas:

```
import { UseGuards } from '@nestjs/common';
import { JwtAuthGuard } from '../auth/jwt-auth.guard';

@UseGuards(JwtAuthGuard)
@Controller('tarefas')
export class TarefasController { /* ... */ }
```

8. Testar no Postman (curl prontos)

No Postman: **Import -> Raw text**, cole um comando abaixo e clique em **Send**. O Postman tem um "cookie jar" próprio: depois do **login**, ele guarda o cookie `access_token` e o envia sozinho nas próximas chamadas (deixe *Send cookies* ligado).

```
# 1) Registrar (201)
curl -X POST http://localhost:3000/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email":"aluno@uncisal.edu.br","senha":"123456"}'
```

```
# 2) Registrar com e-mail inválido (400 - validação)
curl -X POST http://localhost:3000/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email":"xxx","senha":"123456"}'
```

```
# 3) Login (201 - define o cookie httpOnly)
curl -X POST http://localhost:3000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"aluno@uncisal.edu.br","senha":"123456"}'
```

```
# 4) Quem sou eu? (200 - o servidor lê o sub do token no cookie)
curl http://localhost:3000/api/auth/me
```

```
# 5) Listar tarefas - rota protegida (200 logado / 401 sem cookie)
curl http://localhost:3000/api/tarefas
```

```
# 6) Criar tarefa - rota protegida (201)
curl -X POST http://localhost:3000/api/tarefas \
  -H "Content-Type: application/json" \
  -d '{"titulo":"Estudar JWT","descricao":"Cookies e guards"}'
```

```
# 7) Login com senha errada (401)
curl -X POST http://localhost:3000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"aluno@uncisal.edu.br","senha":"errada"}'
```

```
# 8) Logout (limpa o cookie) e depois /me volta a dar 401
curl -X POST http://localhost:3000/api/auth/logout
```

No terminal (fora do Postman) o cookie não é guardado sozinho: use um *cookie jar*. Adicione `-c cookies.txt` no **login** (passo 3) e `-b cookies.txt` nas chamadas protegidas (passos 4, 5, 6). Ex.: `curl -b cookies.txt http://localhost:3000/api/tarefas`

O ponto central: em `GET /api/auth/me` (ou em qualquer rota protegida) o servidor devolve o `userId` (o `sub`) lido do token que está no cookie — o cliente não envia o id; quem garante a identidade é o token validado no servidor.

---

9. Validar pelo frontend

O frontend já tem as telas de **login** e **cadastro** e **protege** a tela de tarefas. Suba tudo e teste no navegador:

```
docker compose up --build        # frontend (5173) + Postgres
# em outro terminal:
cd backend-final &amp;&amp; npm install &amp;&amp; npm run migration:run &amp;&amp; npm run start:dev
```

- Abra http://localhost:5173 -> sem sessão, cai em **/login**.
- **Cadastre-se** (ou entre) -> vai para a lista de tarefas (o cookie foi definido).
- Crie/edite/exclua tarefas (passam pela API protegida, com o cookie).
- **Sair** -> volta para /login; recarregar `/` deslogado também redireciona.